

CUADERNO DE PRACTICAS

UD2 — Procesos del Sistema

Windows Server + Sysinternals Suite

MF0490_3 | IFCT0109 Seguridad Informatica

Entorno principal	DC01 — Windows Server 2019 (10.1.99.60, dominio lab.local)
Acceso	https://juanprofesor.com (Apache Guacamole)
Credenciales	Administrador / Password123! (dominio LAB)
Sysmon	v15.20 preinstalado — config SwiftOnSecurity
Herramientas	PowerShell 5.1 · Task Manager · Sysinternals Suite (ProcExp, ProcMon, Autoruns, PsTools)

Num.	Practica	Herramienta	Nivel	Horas
W-01	Exploración con Task Manager y Get-Process	PowerShell nativo	Basica	1.5h
W-02	Ciclo de vida, prioridades y afinidad de CPU	PowerShell + tasklist/taskkill	Basica	2h
W-03	Servicios: dependencias y hardening	Get-Service + sc.exe	Intermedia	2h
W-04	Process Explorer: DLLs y firmas digitales	Sysinternals ProcExp	Intermedia	2.5h
W-05	Process Monitor: filtrado y traza de eventos	Sysinternals ProcMon	Intermedia	2.5h
W-06	Autoruns: persistencia y deteccion de malware	Sysinternals Autoruns + VT	Avanzada	3h
W-07	PsTools + Sysmon: auditoria forense	PsTools + Sysmon EID 1/7/10	Avanzada	3h

juanprofesor.com | Laboratorio Guacamole

PRÁCTICA W-01

Exploración de Procesos: Task Manager y Get-Process

UD2 — Análisis y Gestión de Procesos · 1.5 horas · Básica

Herramienta principal: Task Manager (GUI) + PowerShell Get-Process

Objetivo

Familiarizarse con las vistas de procesos de Windows Server desde la interfaz gráfica (Task Manager) y desde PowerShell, identificando los parámetros que caracterizan cada proceso: PID, prioridad, usuario propietario, consumo de CPU y memoria.

EN EL LAB

Entorno: DC01 (10.1.99.60, Windows Server 2019). Acceso vía Guacamole → juanprofesor.com.
Credenciales: Administrador / Password123!

PASO 1 — Exploración con Task Manager (GUI)

Abre el Administrador de Tareas con la combinación Ctrl+Shift+Esc o haciendo clic derecho sobre la barra de tareas.

1. Ve a la pestaña Procesos. Observa las columnas: Nombre, PID, Estado, CPU, Memoria.
2. Haz clic en la columna CPU para ordenar por consumo descendente. Anota el proceso con mayor uso.
3. Activa la columna PID si no aparece: clic derecho sobre el encabezado → activar PID.
4. Ve a la pestaña Detalles. Aquí tienes la vista técnica completa con PID, Estado, Usuario y Prioridad base.
5. Haz clic derecho sobre cualquier proceso → ir a Servicios para ver si tiene servicios asociados.
6. Ve a la pestaña Rendimiento → CPU. Observa el número de núcleos lógicos y la curva de carga.

Pregunta 1: ¿Cuántos procesos aparecen en la pestaña Detalles? ¿Cuál es el proceso con mayor consumo de memoria? Anota PID y nombre.

Respuesta:

PASO 2 — Exploración con PowerShell

```
# Abrir PowerShell como Administrador en DC01
# Inicio → PowerShell → Ejecutar como administrador

# Listar todos los procesos (equivalente a ps aux en Linux)
Get-Process

# Ver los 10 procesos con mayor consumo de CPU
Get-Process | Sort-Object CPU -Descending | Select-Object -First 10

# Ver los 10 procesos con mayor consumo de memoria (WorkingSet)
Get-Process | Sort-Object WorkingSet64 -Descending |
  Select-Object Name, Id, CPU,
  @{N='RAM_MB';E={[math]::Round($_.WorkingSet64/1MB,1)}} |
  Format-Table -AutoSize | Select-Object -First 10

# Buscar un proceso específico por nombre
Get-Process -Name 'svchost'

# Ver propiedades completas de un proceso (sustituir PID)
```

```
Get-Process -Id <PID> | Format-List *
```

Pregunta 2: Ejecuta el comando de los 10 procesos con mayor RAM. ¿Qué proceso ocupa más memoria? ¿Cuántos MB consume aproximadamente?

Respuesta:

PASO 3 — Información de propietario y rutas

```
# Ver el usuario propietario de cada proceso (requiere Admin)
Get-WmiObject Win32_Process |
  Select-Object Name, ProcessId,
    @{N='Owner';E={$_.GetOwner().User}} |
  Sort-Object Name | Format-Table -AutoSize

# Ver la ruta del ejecutable de cada proceso
Get-Process | Select-Object Name, Id, Path |
  Where-Object {$_.Path -ne $null} |
  Format-Table -AutoSize

# Buscar procesos sin ruta visible (pueden ser del kernel)
Get-Process | Where-Object {$_.Path -eq $null} |
  Select-Object Name, Id | Format-Table
```

Pregunta 3: ¿Cuántos procesos no tienen ruta de ejecutable visible (Path = null)? ¿A qué se debe esto en Windows?

Respuesta:

PASO 4 — Exportar inventario de procesos

```
# Exportar lista de procesos a CSV para análisis posterior
Get-Process | Select-Object Name, Id, CPU,
  @{N="RAM_MB";E={[math]::Round($_.WorkingSet64/1MB,1)}},
  @{N="Hilos";E={$_.Threads.Count}},
  @{N="Modulos";E={$_.Modules.Count}} |
  Export-Csv -Path "C:\Users\Public\procesos_dc01.csv" -NoTypeInformation

# Verificar que se ha creado el fichero
Get-Item "C:\Users\Public\procesos_dc01.csv" | Select-Object Name, Length, LastWriteTime
```

ENTREGABLE

- Captura de pantalla de la pestaña Detalles del Task Manager con la columna PID visible.
- Captura de la salida del comando 'Top 10 por RAM' en PowerShell.
- Fichero procesos_dc01.csv generado (abrir en Excel y adjuntar captura).
- Respuestas razonadas a las 3 preguntas.

PRÁCTICA W-02

Ciclo de Vida de Procesos y Gestión de Prioridades

UD2 — Análisis y Gestión de Procesos · 2 horas · Básica

Herramienta principal: PowerShell + tasklist + taskkill

Objetivo

Gestionar el ciclo de vida completo de procesos en Windows: creación, modificación de prioridad base, asignación de afinidad de CPU y terminación controlada, tanto desde PowerShell como desde la línea de comandos clásica (CMD).

EN EL LAB

Entorno: DC01 (10.1.99.60). Los comandos de tasklist y taskkill se ejecutan en CMD (símbolo del sistema). Los comandos Get-Process y Start-Process se ejecutan en PowerShell como Administrador.

PASO 1 — Crear y observar un proceso de prueba

```
# PowerShell: lanzar un proceso en segundo plano
# Usamos ping en bucle como proceso de prueba observable
$job = Start-Process -FilePath "ping.exe"
        -ArgumentList "-t 8.8.8.8"
        -WindowStyle Minimized -PassThru

# Guardar el PID para usarlo en los siguientes pasos
Write-Host "PID del proceso ping: " $job.Id
$PID_PING = $job.Id

# Verificar que el proceso existe
Get-Process -Id $PID_PING | Select-Object Name, Id, PriorityClass, CPU
```

Pregunta 1: ¿Qué PriorityClass (prioridad base) tiene el proceso ping por defecto al ser lanzado con Start-Process sin especificar prioridad?

Respuesta:

PASO 2 — Modificar la prioridad del proceso

Las prioridades en Windows tienen 6 niveles. La tabla siguiente muestra su equivalencia con el valor numérico interno:

Nombre (PowerShell)	Nombre (Task Manager)	Valor numérico	Uso típico
Idle	Baja	4	Procesos de mantenimiento, screensavers
BelowNormal	Por debajo de normal	6	Tareas que no deben interferir con el usuario
Normal	Normal	8	Valor predeterminado para la mayoría de procesos
AboveNormal	Por encima de normal	10	Procesos que requieren mayor atención del scheduler

High	Alta	13	Servicios críticos del sistema
RealTime	Tiempo real	24	PELIGROSO — puede bloquear el sistema

```
# Cambiar prioridad a BelowNormal (tarefas no críticas)
$proc = Get-Process -Id $PID_PING
$proc.PriorityClass = [System.Diagnostics.ProcessPriorityClass]::BelowNormal

# Verificar el cambio
Get-Process -Id $PID_PING | Select-Object Name, Id, PriorityClass

# Cambiar prioridad a AboveNormal
$proc.PriorityClass = [System.Diagnostics.ProcessPriorityClass]::AboveNormal
Get-Process -Id $PID_PING | Select-Object Name, Id, PriorityClass

# Verificar también desde Task Manager:
# Pestaña Detalles → clic derecho sobre ping.exe → Establecer prioridad
```

AVISO

NUNCA establezcas prioridad RealTime (Tiempo real) en un proceso de prueba. Puede hacer que el scheduler del kernel quede bloqueado y requiera reinicio del sistema.

PASO 3 — Afinidad de CPU (CPU Affinity)

La afinidad de CPU permite restringir un proceso a ejecutarse solo en determinados núcleos lógicos. DC01 tiene varios núcleos disponibles en la VM.

```
# Ver cuántos procesadores lógicos tiene el sistema
(Get-WmiObject Win32_ComputerSystem).NumberOfLogicalProcessors

# Ver la afinidad actual del proceso (máscara de bits)
(Get-Process -Id $PID_PING).ProcessorAffinity

# Restringir el proceso al núcleo 0 solamente (máscara binaria = 1)
(Get-Process -Id $PID_PING).ProcessorAffinity = 1

# Restringir a núcleos 0 y 1 (máscara binaria = 11 = decimal 3)
(Get-Process -Id $PID_PING).ProcessorAffinity = 3

# Verificar desde Task Manager:
# Detalles → clic derecho → Establecer afinidad
```

Pregunta 2: Si DC01 tiene 4 núcleos lógicos y quieres asignar el proceso a los núcleos 0 y 2, ¿qué valor decimal usarías para ProcessorAffinity? Razona la respuesta con la máscara de bits.

Respuesta:

PASO 4 — Terminación de procesos (PowerShell y CMD)

```
# PowerShell: terminar proceso elegantemente
Stop-Process -Id $PID_PING

# PowerShell: forzar terminación
Stop-Process -Id $PID_PING -Force
```

```
# CMD (símbolo del sistema): equivalente a kill -9
# Terminar por PID
taskkill /PID <PID> /F

# Terminar por nombre de proceso (todos los procesos con ese nombre)
taskkill /IM notepad.exe /F

# tasklist: equivalente a ps aux en CMD
tasklist /V
tasklist /FI "STATUS eq RUNNING"
tasklist /FI "USERNAME eq Administrador"

# Ver árbol de procesos padre-hijo en CMD
wmic process get Name,ProcessId,ParentProcessId /FORMAT:TABLE
```

Pregunta 3: ¿Cuál es la diferencia entre *Stop-Process* sin *-Force* y *Stop-Process -Force*? ¿A qué señales de Linux equivalen respectivamente?

Respuesta:

ENTREGABLE

- Captura de la tabla de prioridades en Task Manager mostrando el proceso ping con prioridad modificada.
- Captura del comando Get-Process mostrando el PriorityClass cambiado a AboveNormal.
- Captura de 'tasklist /V' filtrado por usuario Administrador.
- Respuestas razonadas a las 3 preguntas.

PRÁCTICA W-03

Gestión de Servicios Windows: Dependencias y Hardening

UD2 — Análisis y Gestión de Procesos · 2 horas · Intermedia

Herramienta principal: **Get-Service** + **sc.exe** + **services.msc**

Objetivo

Administrar el ciclo de vida de los servicios de Windows Server, analizar las dependencias entre servicios, identificar servicios innecesarios que amplían la superficie de ataque y aplicar medidas básicas de hardening desactivando servicios no requeridos.

EN EL LAB

Entorno: DC01 (10.1.99.60). Nota: en un DC real en producción NUNCA se deshabilitan servicios sin planificación previa. Esta práctica usa servicios seguros para la demostración.

PASO 1 — Inventario completo de servicios

```
# Ver todos los servicios: nombre, estado y tipo de inicio
Get-Service | Format-Table Name, Status, StartType -AutoSize

# Contar servicios por estado
Get-Service | Group-Object Status | Select-Object Name, Count

# Solo servicios en ejecución
Get-Service | Where-Object {$_.Status -eq 'Running'} |
  Sort-Object Name | Format-Table Name, DisplayName, StartType -AutoSize

# Exportar inventario completo a CSV
Get-Service | Select-Object Name, DisplayName, Status, StartType |
  Export-Csv -Path "C:\Users\Public\servicios_dc01.csv" -NoTypeInformation
```

Pregunta 1: ¿Cuántos servicios hay en estado Running? ¿Cuántos en estado Stopped? ¿Cuántos tienen StartType Disabled?

Respuesta:

PASO 2 — Análisis de dependencias

```
# Ver servicios de los que DEPENDE un servicio dado (DependentServices)
# Ejemplo: servicio Netlogon (clave en un DC)
(Get-Service "Netlogon").DependentServices | Format-Table Name, Status

# Ver de qué servicios DEPENDE Netlogon para arrancar
(Get-Service "Netlogon").RequiredServices | Format-Table Name, Status

# Ver árbol completo de dependencias del servicio DNS
(Get-Service "DNS").DependentServices
(Get-Service "DNS").RequiredServices

# Usando sc.exe (CMD clásico): ver dependencias
sc qc DNS
sc enumdepend DNS 2000
```

```
# Ver dependencias en formato gráfico:
# services.msc → clic derecho sobre DNS → Propiedades → Dependencias
```

Pregunta 2: ¿De qué servicios depende el servicio DNS para poder arrancar? ¿Qué ocurriría si detuvieras el servicio 'RpcSs' (RPC)?

Respuesta:

PASO 3 — Controlar servicios con PowerShell y sc.exe

```
# Detener un servicio
Stop-Service -Name "Spooler" -Force

# Iniciar un servicio
Start-Service -Name "Spooler"

# Reiniciar un servicio
Restart-Service -Name "Spooler"

# Cambiar el tipo de inicio
Set-Service -Name "Spooler" -StartupType Manual
Set-Service -Name "Spooler" -StartupType Automatic

# Con sc.exe (CMD) — más control sobre tipo de inicio
sc config Spooler start= disabled
sc config Spooler start= auto
sc start Spooler
sc stop Spooler
sc query Spooler
```

AVISO

El servicio Spooler (cola de impresión) es seguro de detener en un servidor sin impresoras. Es la superficie de ataque explotada por PrintNightmare (CVE-2021-1675). Deshabilitarlo es una medida de hardening real en DCs.

PASO 4 — Hardening: identificar servicios innecesarios en un DC

Un Controlador de Dominio no debería tener ciertos servicios activos. Comprueba el estado de los siguientes:

Servicio	Nombre	Riesgo si activo en DC	Acción recomendada
Print Spooler	Spooler	PrintNightmare (CVE-2021-1675)	Deshabilitar
Remote Registry	RemoteRegistry	Acceso remoto al registro	Deshabilitar
Telnet	TlntSvr	Transmite credenciales en claro	Deshabilitar / no instalar
SNMP Service	SNMP	Puede revelar información del sistema	Deshabilitar si no se usa
Windows Search	WSearch	No necesario en un DC	Deshabilitar
Xbox Live	XblGameSave	Innecesario en servidor	Deshabilitar

```
# Verificar estado de los servicios de la tabla anterior
```



```
$servicios = @("Spooler","RemoteRegistry","TlntSvr","SNMP","WSearch","XblGameSave")
foreach ($svc in $servicios) {
    $s = Get-Service -Name $svc -ErrorAction SilentlyContinue
    if ($s) {
        Write-Host "$($s.Name): $($s.Status) [$($s.StartType)]"
    } else {
        Write-Host "$svc: NO INSTALADO"
    }
}
```

Pregunta 3: ¿Cuáles de los servicios de la tabla están activos en DC01? Para los que estén Running, ¿qué comandos usarías para deshabilitarlos de forma permanente?

Respuesta:

ENTREGABLE

- Fichero servicios_dc01.csv con el inventario completo.
- Captura del árbol de dependencias del servicio DNS (desde services.msc o PowerShell).
- Captura del script de verificación de servicios de hardening con los resultados obtenidos.
- Respuestas razonadas a las 3 preguntas.

PRÁCTICA W-04

Process Explorer: Árbol de Procesos, DLLs y Verificación de Firmas

UD2 — Análisis y Gestión de Procesos · 2.5 horas · Intermedia

Herramienta principal: Sysinternals Process Explorer (procexp64.exe)

Objetivo

Utilizar Process Explorer para analizar el árbol de procesos padre-hijo, inspeccionar los módulos DLL cargados por cada proceso, verificar la firma digital de ejecutables y detectar procesos con características anómalas (sin firma, rutas inusuales o inyección de DLLs).

SYSINTERNALS

Sysinternals Suite es un conjunto de herramientas avanzadas de diagnóstico para Windows desarrolladas por Mark Russinovich (Microsoft). Descarga: <https://learn.microsoft.com/en-us/sysinternals/downloads/> — O directamente: <https://live.sysinternals.com/procexp64.exe>

PASO 1 — Descarga e instalación de Process Explorer

```
# PowerShell: descargar Process Explorer directamente
$url = "https://live.sysinternals.com/procexp64.exe"
$dest = "C:\Tools\procexp64.exe"
New-Item -Path C:\Tools -ItemType Directory -Force | Out-Null
Invoke-WebRequest -Uri $url -OutFile $dest

# Verificar la descarga
Get-Item $dest | Select-Object Name, Length, LastWriteTime

# Ejecutar Process Explorer como Administrador
Start-Process -FilePath $dest -Verb RunAs

# Aceptar el EULA en el primer arranque
```

NOTA

Process Explorer no requiere instalación. Es un ejecutable portable. Al ejecutarlo por primera vez pedirá aceptar el acuerdo de licencia de Sysinternals. Acepta para continuar.

PASO 2 — Vista de árbol de procesos y colores

Process Explorer muestra los procesos en árbol (padre → hijo), lo que permite entender qué proceso ha creado a cada hijo. Los colores tienen un significado específico:

Color	Significado
Azul claro	Proceso propio (del usuario actual)
Rosa / Rojo	Proceso de servicios del sistema
Verde	Proceso recién creado (durante los últimos segundos)
Rojo	Proceso que acaba de terminar
Violeta	Proceso con imagen empaquetada (packed executable)
Amarillo	Proceso .NET (aplicaciones gestionadas por el CLR)
Gris oscuro	Proceso suspendido

7. Localiza el proceso lsass.exe (Local Security Authority). Anota su PID y proceso padre.

8. Localiza el árbol de svchost.exe. ¿Cuántas instancias hay? ¿Por qué Windows usa múltiples instancias?
9. Activa Options → VirusTotal.com → Check VirusTotal.com para verificar hashes de ejecutables.
10. En la columna VirusTotal, identifica si algún proceso muestra una detección distinta de 0/XX.

Pregunta 1: ¿Cuál es el proceso padre de lsass.exe? ¿Por qué es importante que lsass.exe NUNCA tenga como padre un proceso que no sea wininit.exe?

Respuesta:

PASO 3 — Inspección de DLLs y handles

```
# En Process Explorer (GUI):
# 1. View → Lower Pane View → DLLs
# 2. Selecciona el proceso 'lsass.exe' en el árbol superior
# 3. El panel inferior muestra todas las DLLs cargadas

# Buscar una DLL específica en todos los procesos:
# Ctrl+F → escribir nombre de DLL → Find

# Ver Handles (ficheros, claves de registro, puertos abiertos):
# View → Lower Pane View → Handles
# Selecciona 'System' y observa los handles del kernel
```

11. Selecciona el proceso explorer.exe y observa sus DLLs. Identifica 3 DLLs del sistema (en C:\Windows\System32).
12. Usa Ctrl+F para buscar la DLL 'ntdll.dll'. ¿En cuántos procesos está cargada? ¿Por qué?

Pregunta 2: ¿Qué es la inyección de DLLs (DLL Injection) y cómo podría Process Explorer ayudarte a detectarla? Describe qué buscarías en el panel de DLLs.

Respuesta:

PASO 4 — Verificación de firmas digitales

```
# En Process Explorer: Options → Verify Image Signatures
# Aparecerá la columna 'Verified Signer' en la vista principal

# Procesos legítimos deben mostrar:
# 'Microsoft Corporation' o el fabricante del software

# Señales de alerta (posible malware):
# - '(Not verified)' para ejecutables en System32
# - Ruta inusual (ej: C:\Users\Temp\svchost.exe)
# - Nombre de proceso legítimo pero en ruta incorrecta

# PowerShell: verificar firma de un ejecutable
Get-AuthenticodeSignature -FilePath 'C:\Windows\System32\lsass.exe'
Get-AuthenticodeSignature -FilePath 'C:\Windows\System32\svchost.exe'

# Escanear todos los ejecutables en System32 buscando no firmados
Get-ChildItem C:\Windows\System32*.exe | ForEach-Object {
    $sig = Get-AuthenticodeSignature $_
    if ($sig.Status -ne "Valid") {
        Write-Host "$($_.Name): $($sig.Status)"
    }
}
```

```
}  
}
```

Pregunta 3: ¿Qué resultado obtienes al verificar la firma de `lsass.exe` con `Get-AuthenticodeSignature`? ¿Qué significaría si el `Status` fuera `'NotSigned'`?

Respuesta:

ENTREGABLE

- Captura de Process Explorer mostrando el árbol completo de procesos con colores visibles.
- Captura del panel inferior de DLLs para el proceso `lsass.exe`.
- Captura de la columna 'Verified Signer' visible para todos los procesos.
- Salida del script PowerShell de verificación de firmas en System32.
- Respuestas razonadas a las 3 preguntas.

PRÁCTICA W-05

Process Monitor: Trazado de Actividad en Tiempo Real

UD2 — Análisis y Gestión de Procesos · 2.5 horas · Intermedia

Herramienta principal: Sysinternals Process Monitor (procmon64.exe)

Objetivo

Utilizar Process Monitor para capturar y filtrar en tiempo real toda la actividad de procesos del sistema: accesos al sistema de ficheros, llamadas al registro de Windows, actividad de red y llamadas al sistema. Aplicar filtros para aislar el comportamiento de un proceso concreto y exportar el resultado para análisis forense.

SYSINTERNALS

Process Monitor combina las antiguas herramientas Filemon y Regmon. Captura TODOS los eventos del sistema en tiempo real. En un servidor activo puede generar millones de eventos por minuto — los filtros son imprescindibles.

PASO 1 — Descarga e inicio de Process Monitor

```
# Descargar Process Monitor
$url = "https://live.sysinternals.com/procmon64.exe"
$dest = "C:\Tools\procmon64.exe"
Invoke-WebRequest -Uri $url -OutFile $dest

# Ejecutar como Administrador
Start-Process -FilePath $dest -Verb RunAs

# Al abrir: la captura comienza INMEDIATAMENTE
# Usa Ctrl+E para pausar/reanudar la captura
# Usa Ctrl+X para borrar los eventos acumulados
```

AVISO

En DC01, Process Monitor capturará eventos de todos los servicios del sistema. El número de eventos puede superar los 100.000 en pocos segundos. Pausa la captura (Ctrl+E) inmediatamente y aplica filtros antes de reanudarla.

PASO 2 — Aplicar filtros (imprescindible)

Los filtros en Process Monitor se configuran en Filter → Filter... (Ctrl+L). La sintaxis es: Campo | Condición | Valor | Acción (Include/Exclude).

13. Pausa la captura: Ctrl+E.
14. Borra eventos acumulados: Ctrl+X.
15. Abre el gestor de filtros: Ctrl+L.
16. Añade el filtro: Process Name | is | cmd.exe | Include → Add.
17. Añade el filtro: Process Name | is | powershell.exe | Include → Add.
18. Marca 'Drop filtered events' para no almacenar eventos excluidos en memoria.
19. Pulsa OK y reanuda la captura: Ctrl+E.

```
# Mientras ProcMon está capturando con filtro cmd.exe/powershell.exe,
# abre CMD y ejecuta estos comandos para generar actividad observable:

# En CMD:
dir C:\Windows\System32
type C:\Windows\System32\drivers\setchosts
```

```
reg query HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion

# En PowerShell:
Get-ChildItem C:\Windows\Temp
Get-ItemProperty HKLM:\SYSTEM\CurrentControlSet\Services\DNS
```

Pregunta 1: En la columna 'Operation' de Process Monitor, ¿qué diferencia hay entre ReadFile y QueryOpen? ¿Qué tipo de actividad representa cada uno?

Respuesta:

PASO 3 — Analizar accesos al registro

20. En la barra de herramientas de ProcMon, desactiva los botones de red y proceso (deja solo el icono de registro).
21. Filtra por: Process Name | is | regedit.exe | Include.
22. Abre regedit.exe (Ejecutar → regedit) y navega por HKLM\SOFTWARE\Microsoft.
23. Observa en ProcMon los eventos RegOpenKey, RegQueryValue y RegCloseKey generados.
24. Identifica qué claves de registro consulta regedit al arrancar.

Pregunta 2: ¿Qué tipo de eventos de registro (RegXxx) genera regedit al navegar por el árbol? ¿Por qué es importante monitorizar el acceso al registro desde el punto de vista de la seguridad?

Respuesta:

PASO 4 — Exportar y analizar la traza

```
# En Process Monitor: File → Save (Ctrl+S)
# Formato: CSV (para analizar en Excel) o PML (nativo, para reabrir en ProcMon)

# Guardar como CSV:
# File → Save → Events displayed using current filter → CSV → OK

# Analizar el CSV con PowerShell
$traza = Import-Csv "C:\Users\Public\procmon_traza.csv"

# Contar eventos por tipo de operacion
$traza | Group-Object Operation |
Sort-Object Count -Descending |
Select-Object Name, Count |
Format-Table -AutoSize

# Filtrar solo accesos fallidos (Result = ACCESS DENIED)
$traza | Where-Object {$_.Result -eq "ACCESS DENIED"} |
Select-Object Time, Process, Operation, Path |
Format-Table -AutoSize
```

Pregunta 3: En tu traza exportada, ¿cuántos eventos de tipo 'ACCESS DENIED' encuentras? ¿A qué rutas o recursos hacen referencia? ¿Es normal este comportamiento?

Respuesta:

ENTREGABLE

- Captura de Process Monitor con el filtro activo mostrando actividad de cmd.exe y powershell.exe.
- Captura mostrando eventos de acceso al registro (RegXxx) generados por regedit.
- Fichero procmon_traza.csv exportado (adjuntar o indicar ruta).
- Salida del análisis PowerShell: tabla de eventos por tipo de operación.
- Respuestas razonadas a las 3 preguntas.

PRÁCTICA W-06

Autoruns: Análisis de Persistencia y Detección de Malware

UD2 — Análisis y Gestión de Procesos · 3 horas · Avanzada

Herramienta principal: Sysinternals Autoruns + VirusTotal

Objetivo

Utilizar Autoruns para auditar todos los mecanismos de arranque automático de Windows (registro, carpetas de inicio, tareas programadas, drivers, LSA providers...) y detectar entradas sospechosas que podrían indicar persistencia de malware. Correlacionar los hallazgos con VirusTotal.

SYSINTERNALS

Autoruns es la herramienta definitiva para analizar persistencia en Windows. Enumera TODOS los puntos de arranque automático que puede explotar un atacante — mucho más completo que msconfig o la pestaña Inicio del Task Manager.

PASO 1 — Descarga y primer análisis

```
# Descargar Autoruns
$url = "https://live.sysinternals.com/autoruns64.exe"
$dest = "C:\Tools\autoruns64.exe"
Invoke-WebRequest -Uri $url -OutFile $dest
Start-Process -FilePath $dest -Verb RunAs

# Autoruns carga todos los puntos de autoarranque automáticamente
# Esperar a que termine el escaneo (barra de progreso inferior)

# Activar verificación con VirusTotal:
# Options → Scan Options → Check VirusTotal.com → OK
# Luego: File → Refresh (F5) para re-escanear con VT
```

Autoruns organiza los puntos de autoarranque en pestañas. Las más relevantes para un analista SOC son:

Pestaña	Qué muestra	Relevancia de seguridad
Logon	Entradas de registro Run/RunOnce, carpetas Startup	Alta — mecanismo más común de persistencia
Services	Servicios de Windows registrados	Alta — malware frecuentemente instala servicios
Scheduled Tasks	Tareas programadas del sistema y usuario	Alta — usado para persistencia y escalada
Drivers	Controladores del kernel cargados	Crítica — drivers maliciosos tienen privilegio SYSTEM
LSA Providers	Proveedores de autenticación LSA	Crítica — persistencia con acceso a credenciales
Boot Execute	Programas ejecutados antes del inicio de sesión	Alta — difícil de detectar sin esta herramienta
Winlogon	Notificadores de eventos de inicio de sesión	Alta — gancho en el proceso de autenticación

PASO 2 — Filtrar entradas sospechosas

25. En Autoruns: Options → Filter Options → Hide Microsoft Entries. Esto elimina las entradas firmadas por Microsoft, dejando solo las de terceros y las potencialmente sospechosas.
26. Adicionalmente activa: Options → Filter Options → Hide VirusTotal Clean Entries (si has activado VT).
27. Revisa las entradas que quedan — estas son las que merecen análisis.
28. Las entradas con fondo AMARILLO indican que el fichero referenciado NO EXISTE (posible traza de malware ya eliminado).
29. Las entradas con fondo ROJO indican que VirusTotal ha detectado el fichero como malicioso.

Pregunta 1: Tras filtrar las entradas de Microsoft, ¿cuántas entradas de terceros quedan en la pestaña Logon? Lista los nombres de los ejecutables y sus rutas.

Respuesta:

PASO 3 — Simular una entrada de persistencia y detectarla

```
# PowerShell: crear una entrada de autoarranque en el registro
# (simulamos la técnica T1547.001 de MITRE ATT&CK)

# Añadir entrada en HKCU Run (se ejecuta al iniciar sesión el usuario actual)
New-ItemProperty -Path "HKCU:SoftwareMicrosoftWindowsCurrentVersionRun"
-Name "SimulacionPersistencia"
-Value "C:Toolsprocexp64.exe"
-PropertyType String -Force

# Verificar la entrada creada
Get-ItemProperty "HKCU:SoftwareMicrosoftWindowsCurrentVersionRun"

# En Autoruns: F5 para refrescar
# La nueva entrada aparece en la pestaña Logon
# Verificar que aparece con el nombre 'SimulacionPersistencia'
```

AVISO

Esta simulación usa un ejecutable legítimo (procexp64.exe). En un escenario real de malware, el valor apuntaría a un ejecutable malicioso. Elimina la entrada al terminar la práctica (Paso 3, bloque de limpieza).

```
# Limpiar: eliminar la entrada de persistencia creada
Remove-ItemProperty -Path "HKCU:SoftwareMicrosoftWindowsCurrentVersionRun"
-Name "SimulacionPersistencia"

# Verificar que se ha eliminado
Get-ItemProperty "HKCU:SoftwareMicrosoftWindowsCurrentVersionRun"

# En Autoruns: F5 para confirmar que la entrada ya no aparece
```

Pregunta 2: ¿En qué clave de registro has creado la entrada de persistencia? ¿Cuál es la diferencia entre HKCU (HKEY_CURRENT_USER) y HKLM (HKEY_LOCAL_MACHINE) para una entrada de autoarranque en términos de persistencia y privilegios?

Respuesta:

PASO 4 — Exportar y analizar con PowerShell

```
# Autoruns tiene versión de línea de comandos: autorunsc64.exe
$url = "https://live.sysinternals.com/autorunsc64.exe"
$dest = "C:Toolsautorunsc64.exe"
Invoke-WebRequest -Uri $url -OutFile $dest

# Exportar TODOS los autoarranques a CSV
C:Toolsautorunsc64.exe -a * -c -h -s * > C:UsersPublicautoruns_dc01.csv

# Analizar con PowerShell: buscar entradas sin firma digital
$ar = Import-Csv "C:UsersPublicautoruns_dc01.csv"
$ar | Where-Object {$_.Signer -eq ""} |
    Select-Object Entry, Category, Description, ImagePath |
    Format-Table -AutoSize

# Contar entradas por categoría
$ar | Group-Object Category |
    Sort-Object Count -Descending |
    Select-Object Name, Count | Format-Table
```

Pregunta 3: ¿Cuántas entradas sin firma digital (Signer vacío) aparecen en tu exportación? Analiza una de ellas: ¿es legítima o sospechosa? ¿Cómo lo determinarías en una investigación real?

Respuesta:

ENTREGABLE

- Captura de Autoruns con la pestaña Logon visible, filtro 'Hide Microsoft' activado.
- Captura de Autoruns mostrando la entrada 'SimulacionPersistencia' antes de eliminarla.
- Fichero autoruns_dc01.csv exportado con autorunsc64.
- Salida del análisis PowerShell: entradas sin firma y conteo por categoría.
- Respuestas razonadas a las 3 preguntas.

PRÁCTICA W-07

PsTools + Sysmon: Auditoria Forense de Procesos

UD2 — Análisis y Gestión de Procesos · 3 horas · Avanzada

Herramienta principal: Sysinternals PsTools (PsList, PsKill, PsExec) + Sysmon v15

Objetivo

Utilizar las herramientas PsTools de Sysinternals para gestión avanzada de procesos locales y remotos, y analizar los eventos de Sysmon para rastrear la creación de procesos, conexiones de red y carga de drivers con fines de auditoría forense.

EN EL LAB

Entorno: DC01 (10.1.99.60). Sysmon v15.20 ya está instalado en DC01 con la configuración SwiftOnSecurity. Los eventos se recogen en el Event Log: Applications and Services Logs → Microsoft → Windows → Sysmon → Operational.

PASO 1 — PsTools: descarga y primer uso

```
# Descargar PsTools Suite completa
$url = "https://download.sysinternals.com/files/PSTools.zip"
$dest = "C:ToolsPSTools.zip"
Invoke-WebRequest -Uri $url -OutFile $dest
Expand-Archive -Path $dest -DestinationPath C:ToolsPSTools -Force

# PsList: lista de procesos (más detallado que tasklist)
C:ToolsPSToolspstlist.exe

# PsList con vista de árbol
C:ToolsPSToolspstlist.exe -t

# PsList: ver un proceso específico y sus hilos
C:ToolsPSToolspstlist.exe -x svchost
```

Pregunta 1: Compara la salida de 'pslist.exe -t' con 'Get-Process | Format-Table'. ¿Qué información adicional proporciona pslist que no aparece en Get-Process?

Respuesta:

PASO 2 — PsKill y PsExec

```
# PsKill: terminar procesos por nombre o PID
# (más seguro que taskkill /F para servicios con handles abiertos)
C:ToolsPSToolspskill.exe notepad

# PsExec: ejecutar proceso en contexto de SYSTEM
# (permite analizar permisos y contexto de seguridad del kernel)
C:ToolsPSToolspsexec.exe -s cmd.exe
# Dentro del CMD resultante: whoami → debe responder 'nt authoritysystem'
# Exit para cerrar la sesión SYSTEM

# PsExec: ejecutar un comando y capturar la salida
C:ToolsPSToolspsexec.exe -s powershell.exe -Command whoami

# Ver información del sistema con SYSTEM context
C:ToolsPSToolspsinfo.exe
```

IMPORTANTE

PsExec con el flag -s otorga privilegios SYSTEM — el nivel más alto de Windows. Usar con extrema precaución. En entornos reales, PsExec es frecuentemente usado por atacantes (Lateral Movement, T1569.002). Sysmon registra TODOS los usos de PsExec.

Pregunta 2: Cuando ejecutas 'psexec.exe -s cmd.exe', Sysmon registra el evento. ¿Qué Event ID de Sysmon corresponde a la creación de un nuevo proceso? ¿Qué campos del evento te permitirían identificar que se usó PsExec?

Respuesta:

PASO 3 — Sysmon: consulta de eventos de proceso

Sysmon registra eventos detallados en el canal de eventos de Windows. Los más relevantes para la UD2 son:

Event ID	Nombre del evento	Información capturada
1	Process Create	Proceso creado: CommandLine, ParentImage, User, Hashes, GUID
5	Process Terminated	Proceso terminado: ProcessGuid, Image, UtcTime
7	Image Loaded	DLL/EXE cargado: ImageLoaded, Signed, Signature, Hashes
8	CreateRemoteThread	Inyección de hilo remoto — indicador de proceso injection
10	ProcessAccess	Acceso entre procesos — típico de credential dumping (Isass)
25	Process Tampering	Modificación de imagen de proceso en memoria (process hollowing)

```
# PowerShell: consultar eventos Sysmon de creación de proceso
$eventos = Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational"
-FooterXPath '[System[EventID=1]]' -MaxEvents 50

# Mostrar los últimos 10 procesos creados
foreach ($e in $eventos | Select-Object -First 10) {
    $xml = [xml]$e.ToXml()
    $data = $xml.Event.EventData.Data
    $img = ($data | Where-Object {$_.Name -eq 'Image'}).'#text'
    $cmd = ($data | Where-Object {$_.Name -eq 'CommandLine'}).'#text'
    $usr = ($data | Where-Object {$_.Name -eq 'User'}).'#text'
    Write-Host "[$usr] $img"
    Write-Host "  CMD: $cmd"
}
```

PASO 4 — Detectar el uso de PsExec en el log de Sysmon

```
# Buscar eventos Sysmon relacionados con PsExec
# PsExec crea el servicio PSEXESVC y lo instala temporalmente
$eventos = Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational"
-FooterXPath '[System[EventID=1]]'

foreach ($e in $eventos) {
    $xml = [xml]$e.ToXml()
    $data = $xml.Event.EventData.Data
```

```

$cmd = ($data | Where-Object {$_.Name -eq 'CommandLine'}).'#text'
$img = ($data | Where-Object {$_.Name -eq 'Image'}).'#text'
if ($cmd -match "psexec|PSEXESVC" -or $img -match "psexec") {
    Write-Host "ALERTA PsExec detectado:"
    Write-Host "  Imagen: $img"
    Write-Host "  Comando: $cmd"
}
}

# Ver también Event ID 7045 (instalación de nuevo servicio)
Get-WinEvent -LogName System -FilterXPath "[System[EventID=7045]]" |
    Where-Object {$_.Message -match 'PSEXESVC'} |
    Select-Object TimeCreated, Message | Format-List

```

Pregunta 3: ¿Encuentras en el log de Sysmon el evento correspondiente al uso de PsExec que realizaste en el Paso 2? Describe los campos Image, CommandLine y User del evento. ¿Por qué este nivel de telemetría es crítico para un analista SOC?

Respuesta:

PASO 5 — Exportar telemetría Sysmon a CSV

```

# Exportar los últimos 200 eventos de creación de procesos a CSV
$eventos = Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational"
            -FilterXPath "[System[EventID=1]]" -MaxEvents 200

$salida = foreach ($e in $eventos) {
    $xml = [xml]$e.ToXml()
    $data = $xml.Event.EventData.Data
    [PSCustomObject]@{
        Tiempo      = $e.TimeCreated
        Usuario      = ($data | Where-Object {$_.Name -eq 'User'}).'#text'
        Proceso      = ($data | Where-Object {$_.Name -eq 'Image'}).'#text'
        CommandLine = ($data | Where-Object {$_.Name -eq 'CommandLine'}).'#text'
        ParentImage = ($data | Where-Object {$_.Name -eq 'ParentImage'}).'#text'
        Hashes       = ($data | Where-Object {$_.Name -eq 'Hashes'}).'#text'
    }
}

$salida | Export-Csv -Path "C:\Users\Public\sysmon_procesos.csv" -NoTypeInformation
Write-Host "Exportados $($salida.Count) eventos a sysmon_procesos.csv"

```

ENTREGABLE

- Captura de 'pslist.exe -t' mostrando el árbol de procesos.
- Captura de la sesión 'psexec.exe -s cmd.exe' mostrando 'whoami → nt authority\system'.
- Captura de la salida del script PowerShell que lista los últimos 10 procesos creados (Sysmon EID 1).
- Salida del script de detección de PsExec en el log de Sysmon.
- Fichero sysmon_procesos.csv exportado (adjuntar o mostrar las primeras 10 filas en captura).
- Respuestas razonadas a las 3 preguntas.

